KDAB

RocketChatQml Architecture

Sergio Martins, 21/10/2026

Trusted Software Excellence since 1999

KDAB

Overview

Trusted Software Excellence since 1999

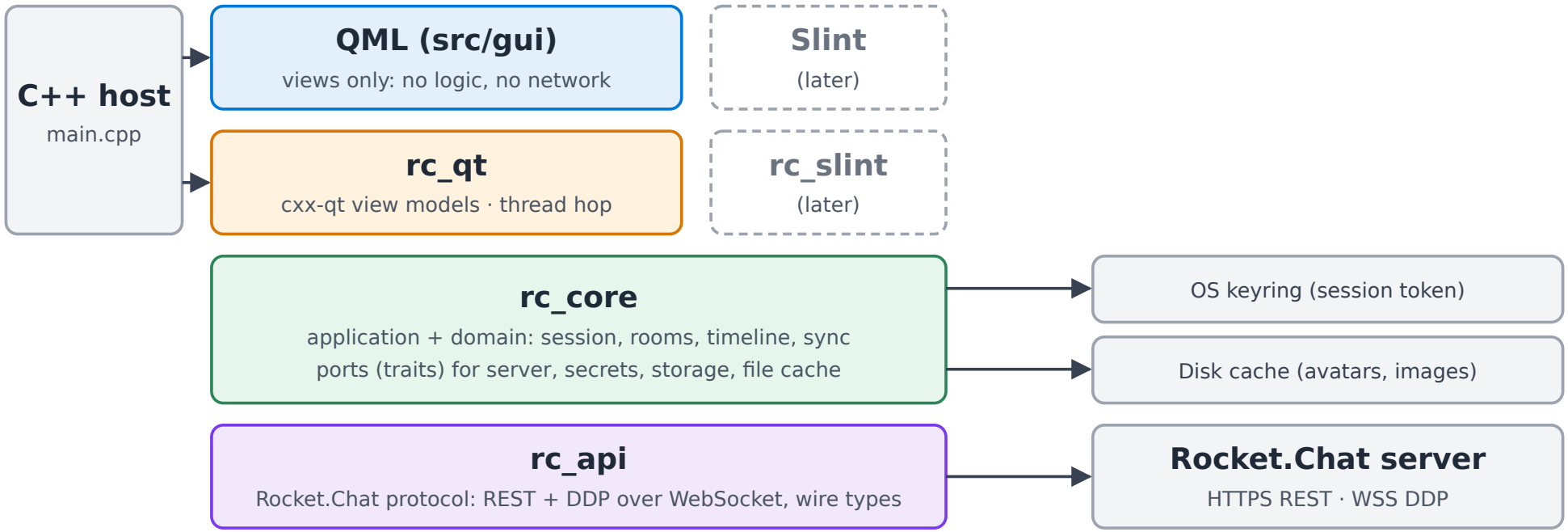
KDAB

What it is

- A Rocket.Chat desktop client
- All logic in **Rust**, with no GUI dependency
- First frontend: **QML**, bound to Rust through **cxx-qt**
- A **Slint** frontend comes later and reuses everything below the GUI layer
- QML is presentation only: it binds to view models and calls their invokables
- The GUI thread never blocks and never touches the network

KDAB

Layers



Dependencies only point downwards · nothing below rc_qt knows about Qt or Slint · dashed = planned

KDAB

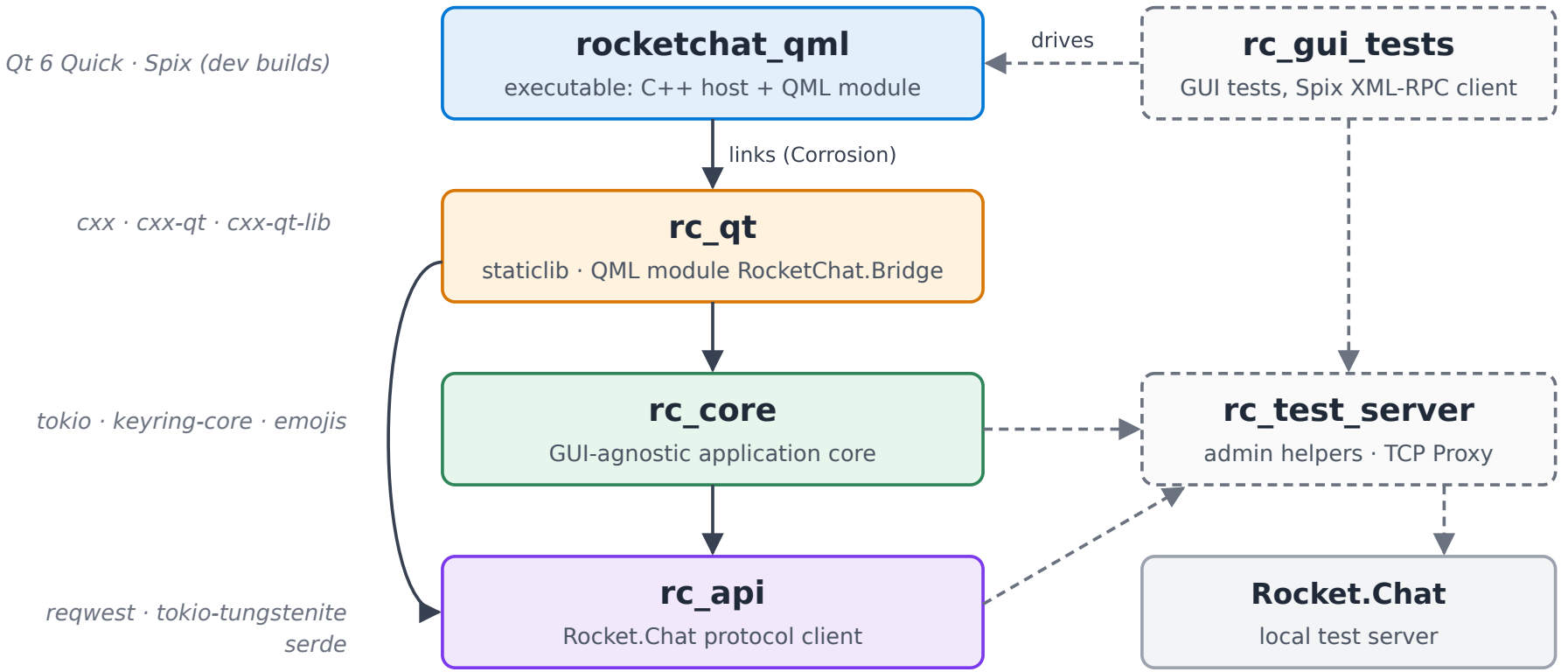
Crates

KDAB

The workspace

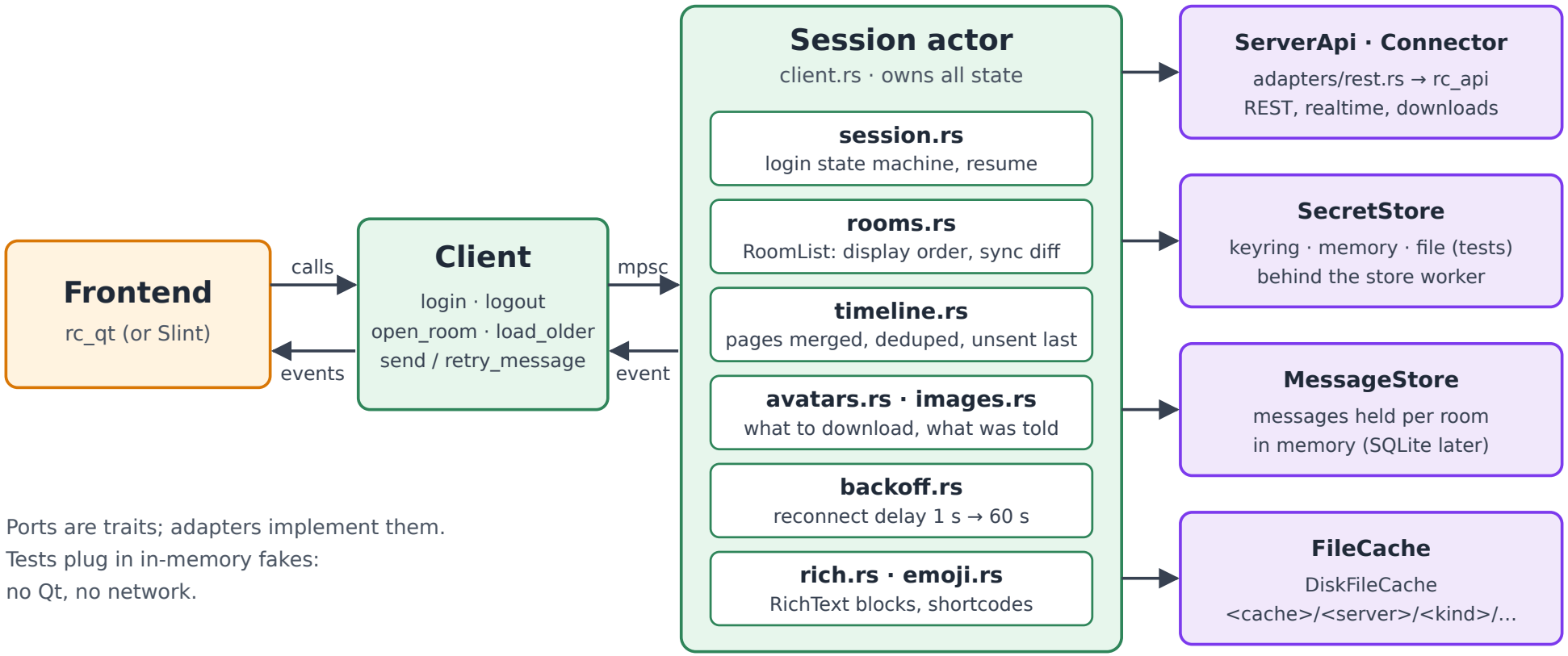
Crate	What it does
`rc_api`	Typed client for Rocket.Chat REST and realtime DDP over WebSocket. Wire types, no state
`rc_core`	Domain types, session actor, sync engine, use cases. Ports (traits) and their adapters
`rc_qt`	cxx-qt `QObject`s and list models for QML. The only crate with `unsafe`
`rc_gui_tests`	GUI tests driving the app over Spix XML-RPC; same paths for QML and Slint
`rc_test_server`	Test helpers: admin API of the local test server, TCP proxy to cut connections
`src/gui`	C++ host (`main.cpp`) and the QML module

Who depends on whom



solid: dependency · dashed: dev-dependency or test-time · grey italics: main third-party crates

rc_core: one actor, pure modules, ports



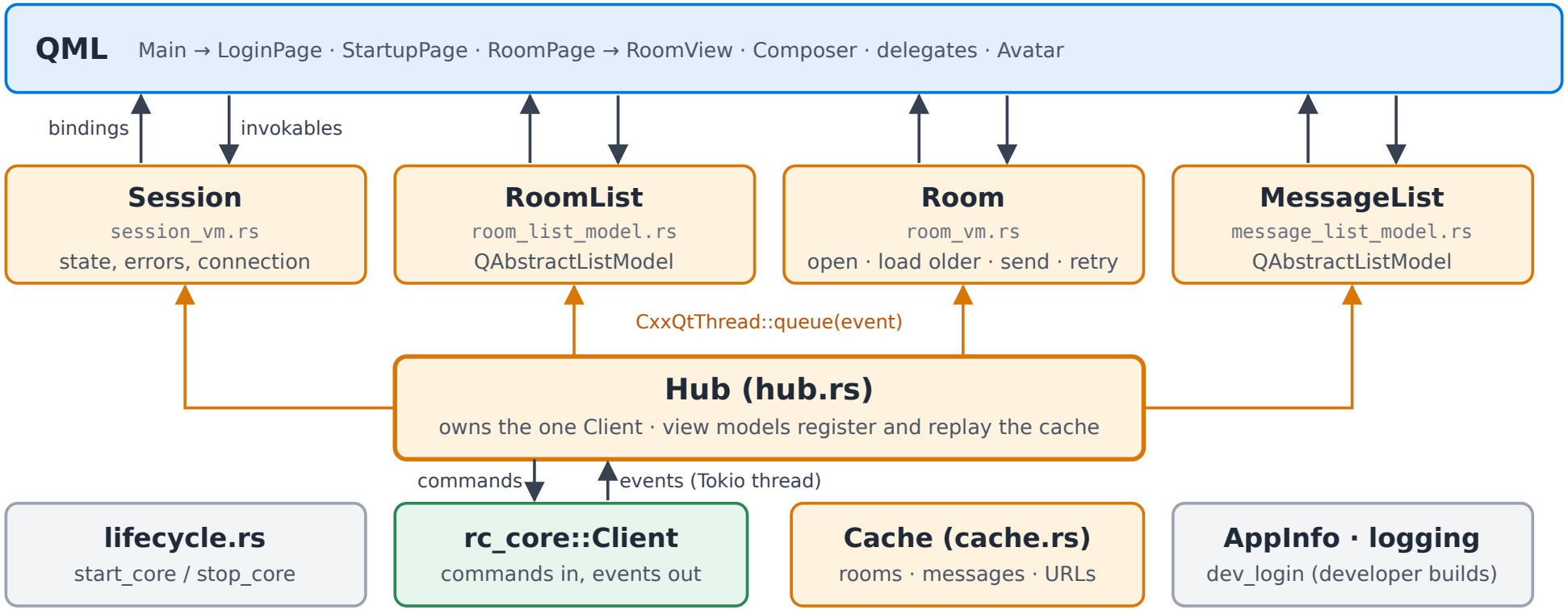
Ports are traits; adapters implement them.
Tests plug in-memory fakes:
no Qt, no network.

KDAB

rc_core: events are index-based diffs

- `Event::Rooms` → insert, update, move, remove
- `Event::Timeline` → reset, prepend, append, insert, update, move
- Also `Session`, `Connection`, `TimelineStatus`, `CustomEmoji`, `Avatars`, `Images`
- View models apply each change **one to one**: no list is rebuilt after a reconnect
- Protocol quirks stay in `rc_api`; `rc_core` maps wire types to domain types
- Message bodies are `RichText` blocks; frontends never see the server's markdown tree

rc_qt: view models around a hub

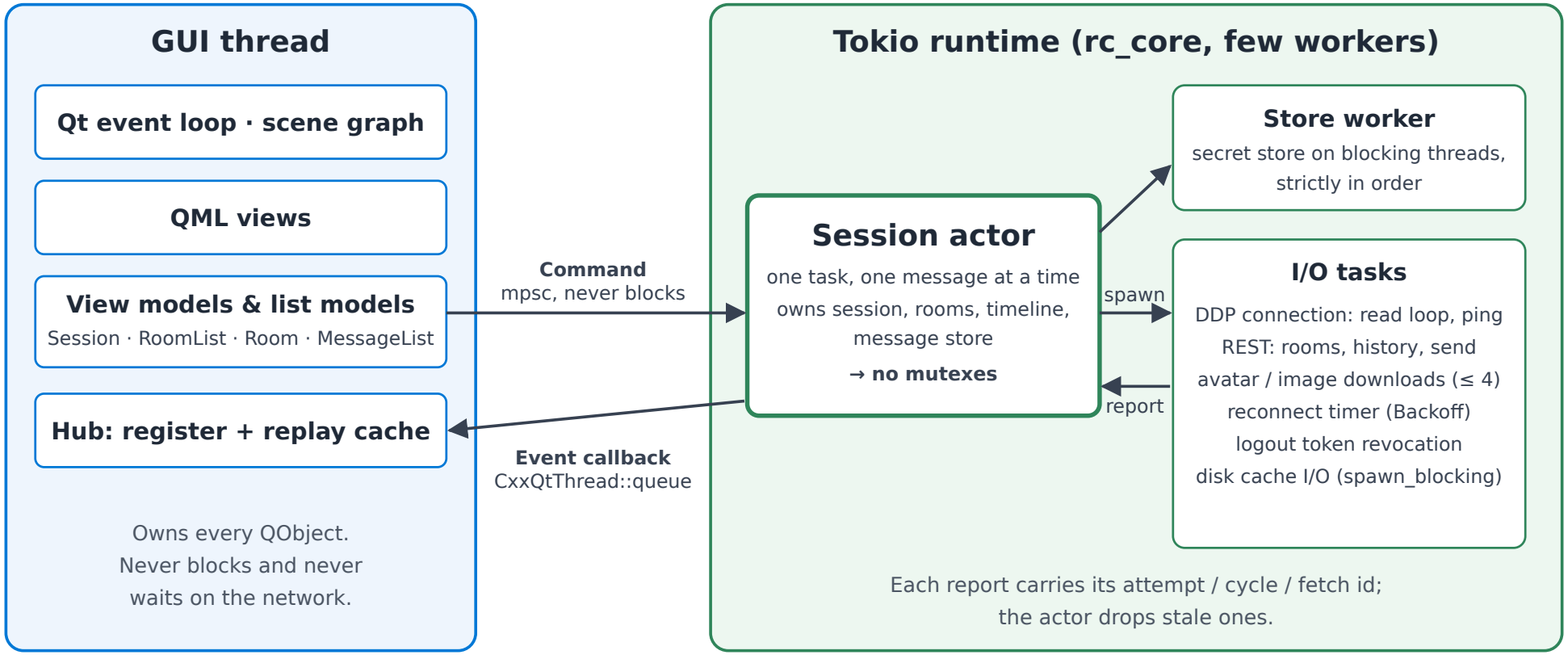


Pure Rust, unit-tested without Qt: cache · row_ops · session_view · room_view · message_view · rich_html · avatar_view · attachment_view

KDAB

Threading

Two worlds, two channels



KDAB

Rules

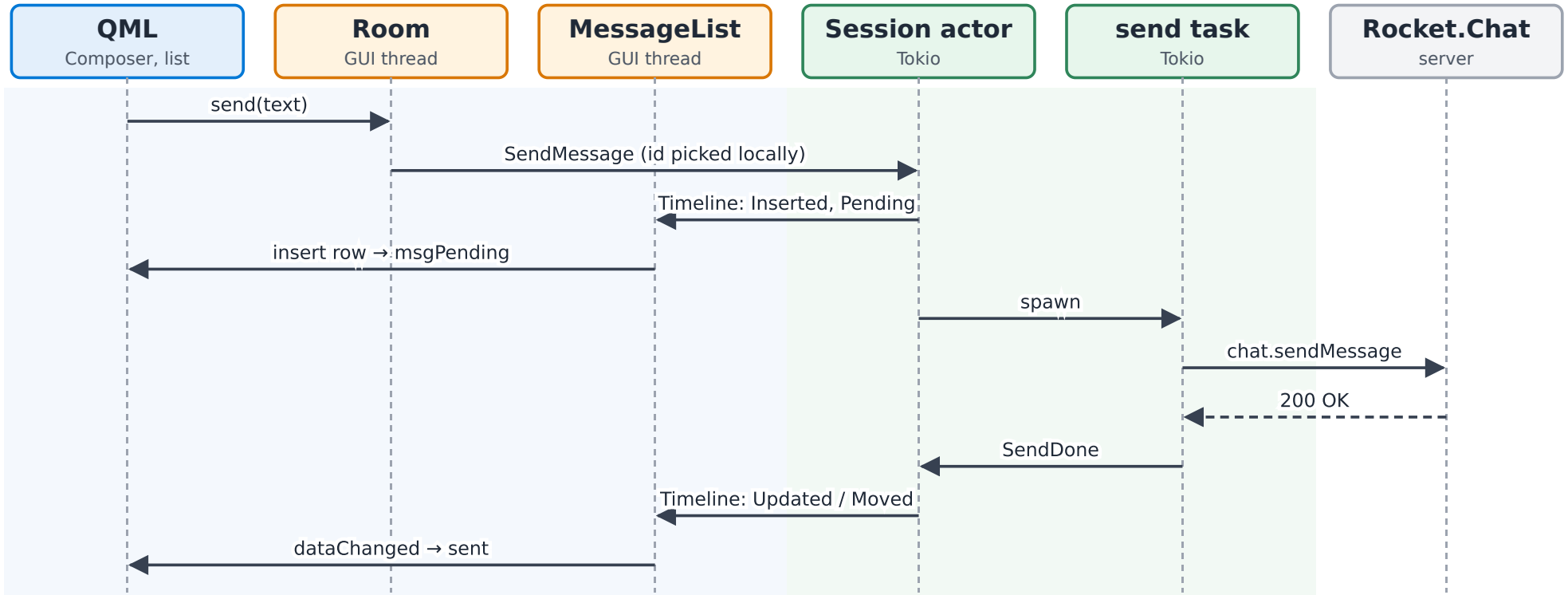
- Qt objects are **never** touched from a Tokio thread
- GUI → core: commands on a non-blocking `mpsc` channel
- Core → GUI: the actor calls the event callback in order; the hub queues each event to the view models with `CxxQtThread::queue`
- A view model created late replays the hub's cache under the same lock: it misses nothing and applies nothing twice
- Blocking work (keyring, disk) runs on blocking threads, never on the async workers
- Shutdown is the only wait: at most 2 s, for a pending logout

KDAB

Data flow

KDAB

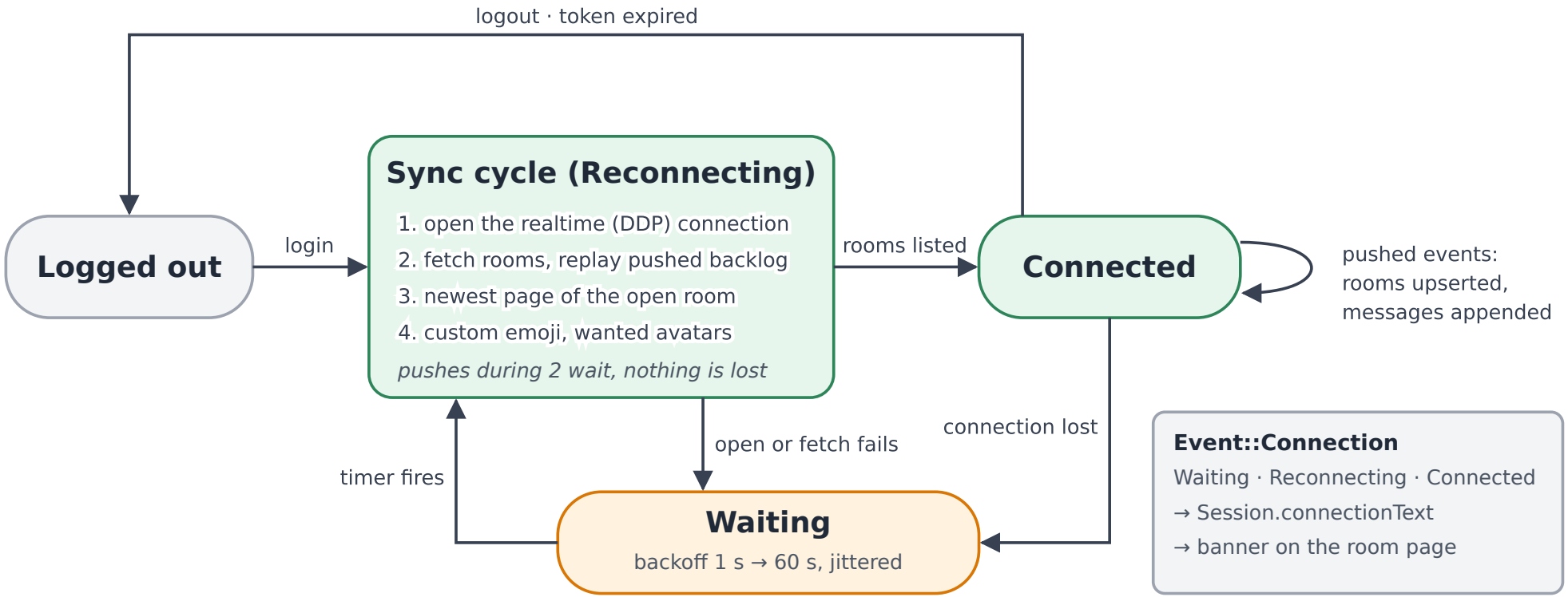
Sending a message



The realtime echo may settle the message first; whichever arrives second finds nothing left to do.
A retry reuses the id, so the server never stores the message twice.

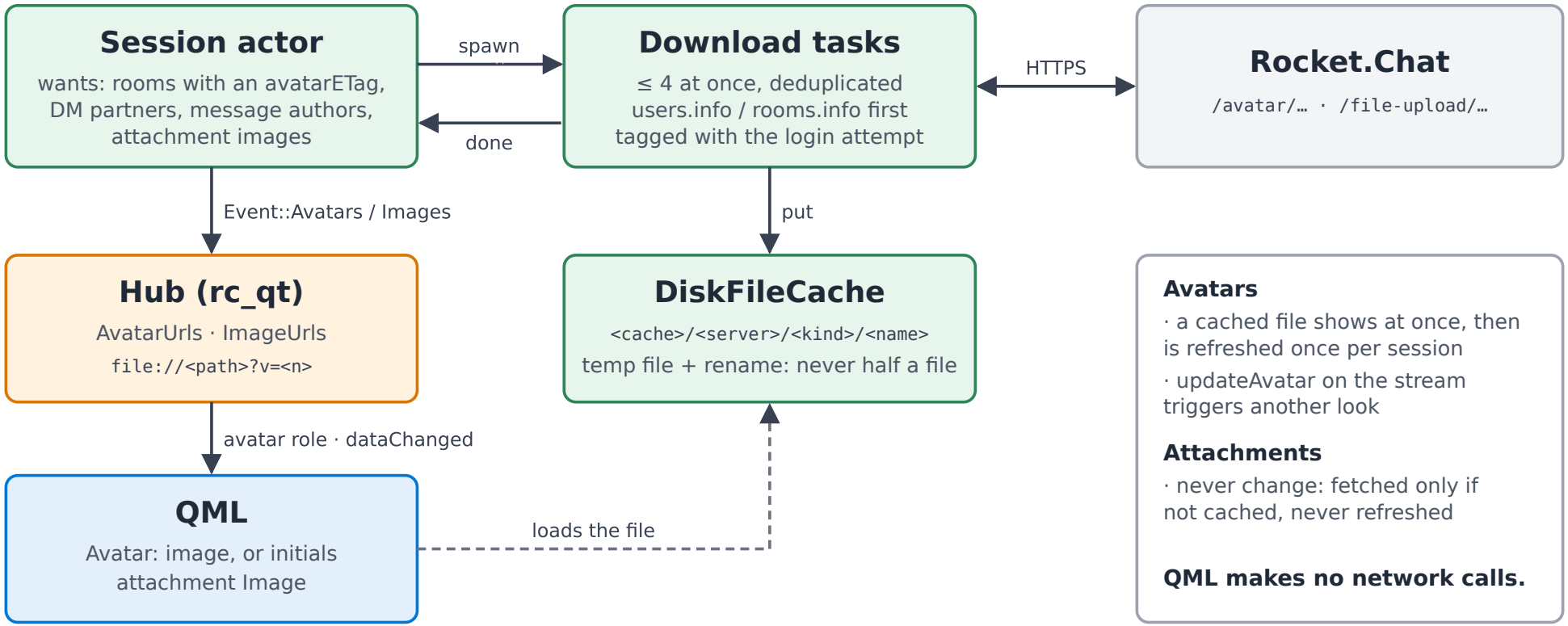
KDAB

Staying in sync



KDAB

Avatars and attachment images



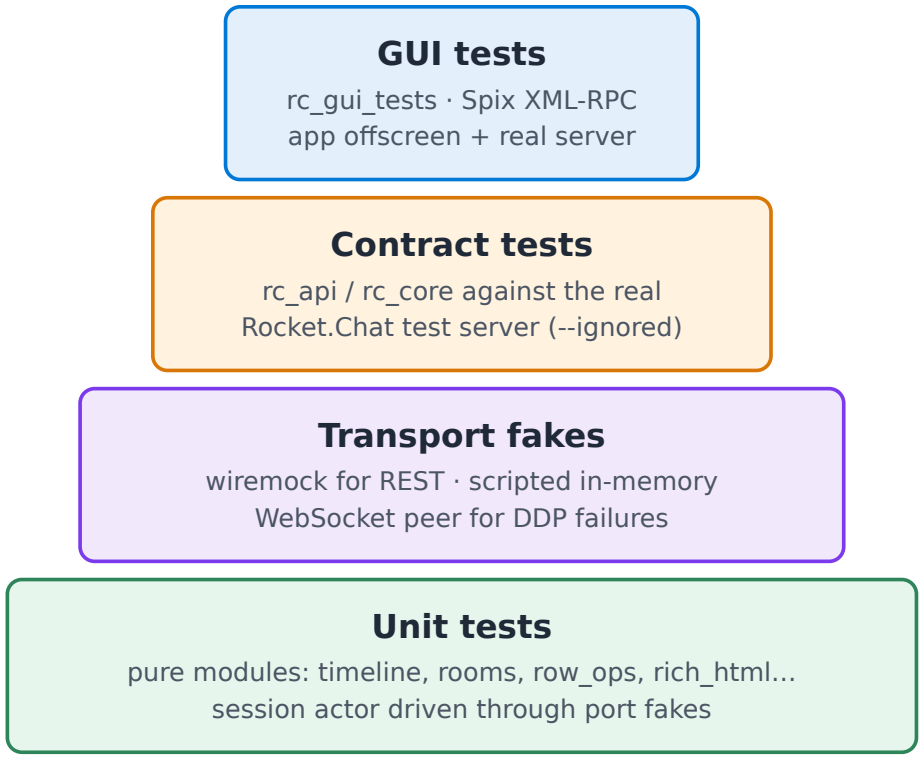
KDAB

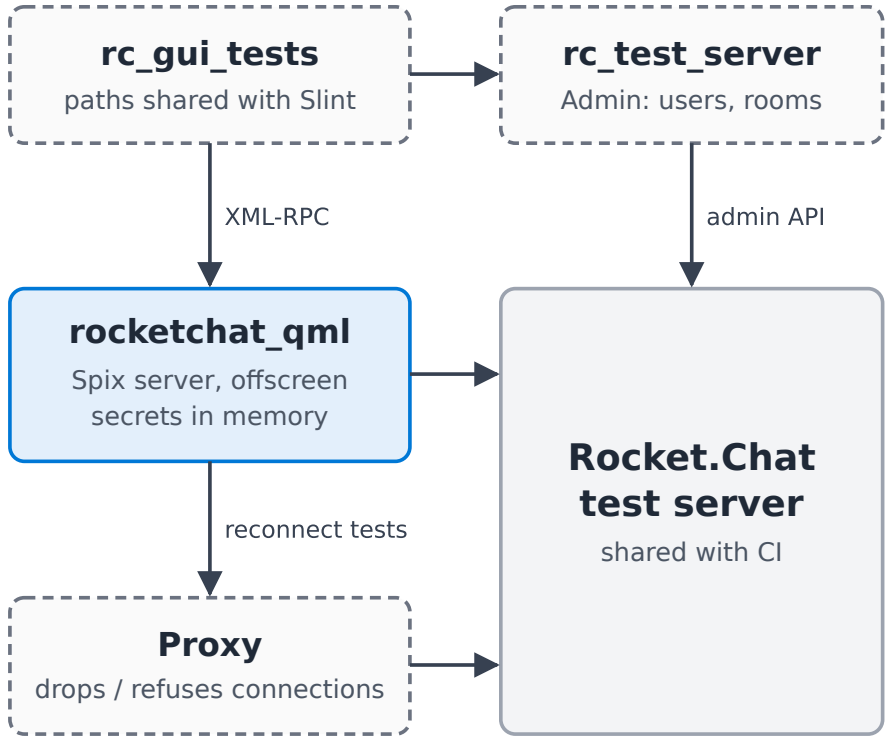
Testing

Trusted Software Excellence since 1999

KDAB

Test levels and the GUI test setup





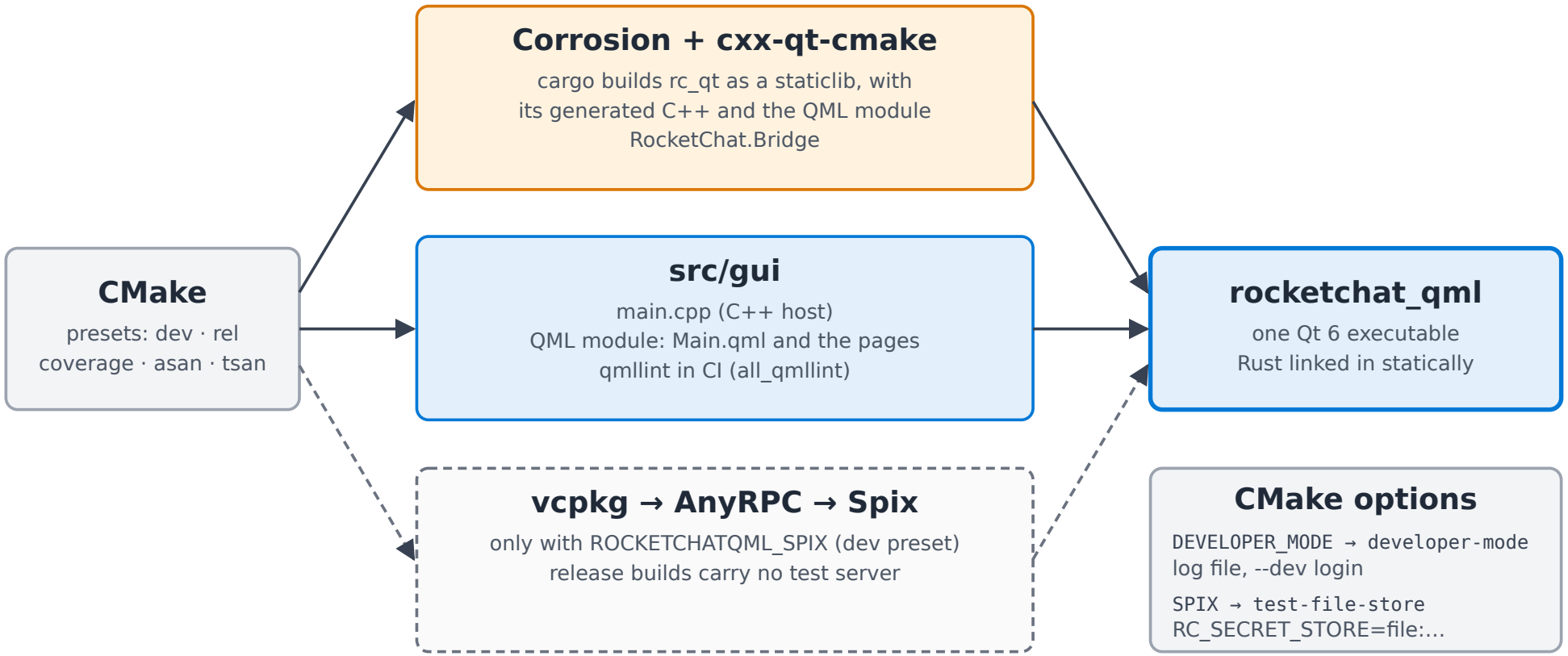
No fake Rocket.Chat: whatever the real server can do is tested against it.

KDAB

Build

Trusted Software Excellence since 1999

From CMake to one executable



KDAB

More detail

- `ARCHITECTURE.md` : the design, in full
- `PLAN.md` : roadmap and progress
- `TRACEABILITY.md` : features and the tests that cover them
- `CXX-QT.md` : cxx-qt limitations and our workarounds